

Tipos de SQL JOINS: Conectando los Datos

Proporcionar una guía visual y técnica para entender cómo relacionar información entre múltiples tablas de bases de datos relacionales.

Contexto

Esta infografía sirve como una referencia educativa detallada sobre las cláusulas JOIN en SQL, esenciales para el desarrollo de software y el análisis de datos. Explica cómo estas herramientas permiten reconstruir información dividida lógicamente en una base de datos normalizada, evitando la duplicidad y manteniendo la integridad de los registros.

Datos del Autor: Patricio Fernando Panales Nolasco | Grupo: 3DSM-Q1.

¿Qué es un JOIN? El Puente de los Datos

Tabla 1		
●	●	■
●	■	■
■	●	■

JOIN

Tabla 2		
●	●	■
●	●	■
■	●	■

Es una cláusula en SQL que combina filas de dos o más tablas basándose en una columna relacionada (generalmente claves primarias y foráneas) para reconstruir información dividida lógicamente, evitando la duplicidad de datos.

Los 6 Tipos Principales de JOINS

INNER JOIN

Retorna filas con coincidencias en ambas tablas. Es el tipo por defecto (JOIN = INNER JOIN).

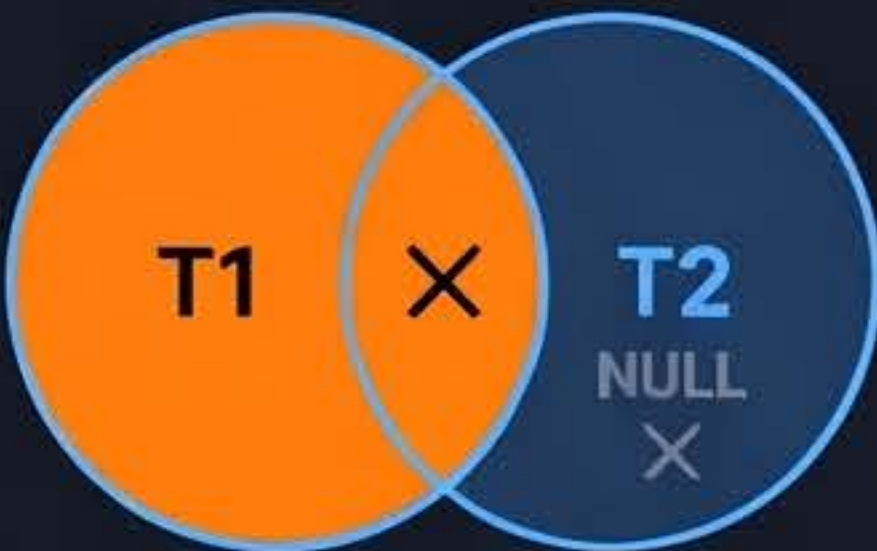
```
SELECT * FROM t1
INNER JOIN t2
ON t1.col = t2.col;
```



LEFT JOIN (LEFT OUTER JOIN)

Retorna todas las filas de la tabla izquierda y las coincidencias de la derecha; llena con NULL si no hay coincidencia.

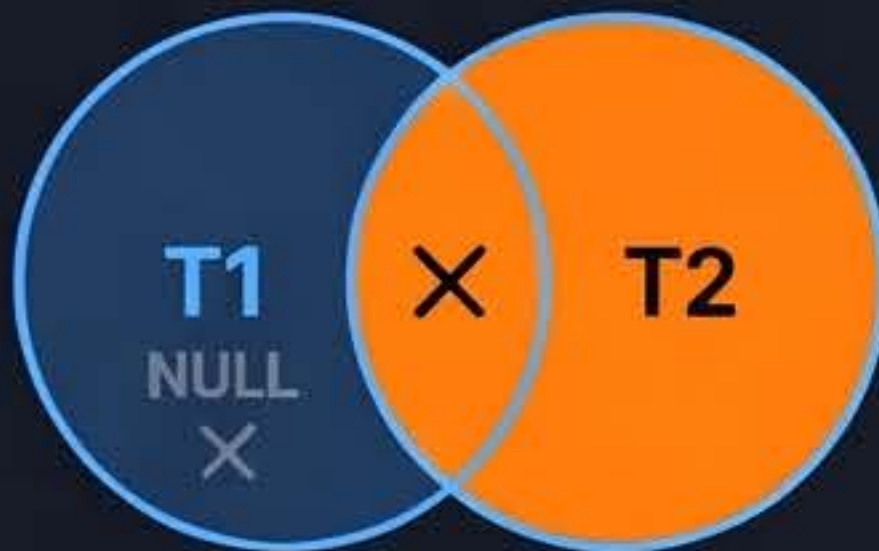
```
SELECT * FROM t1
LEFT JOIN t2
ON t1.col = t2.col;
```



RIGHT JOIN (RIGHT OUTER JOIN)

Retorna todas las filas de la tabla derecha y las coincidencias de la izquierda; llena con NULL si no hay coincidencia.

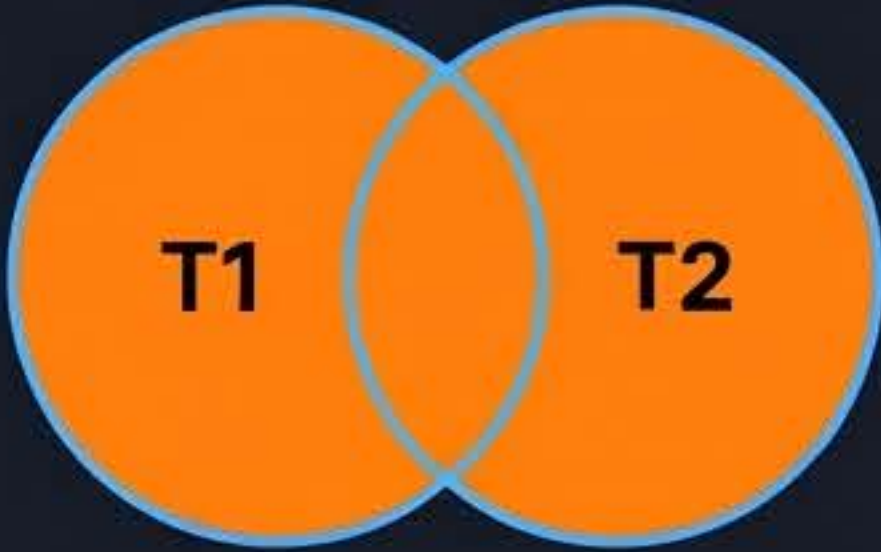
```
SELECT * FROM t1
RIGHT JOIN t2
ON t1.col = t2.col;
```



FULL OUTER JOIN

Retorna todas las filas de ambas tablas, relacionando con NULL donde no existan coincidencias. (En MySQL se simula con UNION de LEFT y RIGHT).

```
SELECT * FROM t1
FULL OUTER JOIN t2
ON t1.col = t2.col;
```



CROSS JOIN

Produce el producto cartesiano; combina cada fila de la primera tabla con cada fila de la segunda. No usa cláusula ON.

```
SELECT * FROM t1
CROSS JOIN t2;
```

T1	
P1	●
P2	●

T2	
●	P3
●	P4

SELF JOIN

Una tabla que se une a sí misma usando alias. Util para jerarquías como empleados y sus supervisores.

```
SELECT e.name, m.name
FROM employees e
LEFT JOIN employees m
ON e.manager_id = m.id;
```

Employees	
m.name	m.name
e.name	

Ejemplo Visual Aplicado

Conjunto de Datos Base

Employees (izq)

Priya	10
Rahul	20
Amit	10
Sara	30
Jason	NULL

Departments (Der)

10	Engineering
20	Sales
30	Marketing
40	Finance

INNER JOIN

Priya	Engineering
Rahul	Sales

Jason y Finanzas quedan fuera (no hay match).

LEFT JOIN

Priya	Engineering
Rahul	Sales
Amit	Marketing
Sara	Finance
Jason	NULL

Muestra a todos los empleados, sin importar el departamento.

RIGHT JOIN

10	Engineering
20	Sales
30	Marketing
40	Finance
	NULL

Muestra todos los departamentos, aunque no tengan empleados.

Puntos Clave y Mejores Prácticas



Rendimiento mediante Índices: Los índices en claves primarias y foráneas son fundamentales para el rendimiento del JOIN en tablas grandes.



Precisión en SELECT: Evita usar 'SELECT *' para reducir el volumen de datos y evitar errores de ambigüedad de columnas.



Filtrado Inteligente: Filtrar un LEFT JOIN en el WHERE puede convertirlo en INNER JOIN. Coloca los filtros en la cláusula ON.

Conclusión y Fuentes

Dominio de JOINS: Dominar los JOINS es indispensable para el desarrollo de software, permitiendo recuperar información compleja de forma eficiente.

Bibliografía Consultada: Documentación oficial de SQL, SQLLabHub (2026), DEV Community, GeeksforGeeks, SQL Practice Online, W3Schools.